

# MÓDULO 13: Introducción al Desarrollo de Software con Agentes en Antigravity-IDE

**Curso:** Python para Ciencia Abierta (CSIC)

**Autor:** Gustavo Liñán Cembrano

**Fechas:** 5 a 8 de Octubre de 2026

**Tiempo Estimado de Impartición:** 1 hora

**Módulo:** 13 - Paradigma de Desarrollo con Agentes IA, SDK de Gemini ( `google-generativeai` ), Bucle Percepción-Acción-Verificación (PAV), Reglas ( `AGENTS.md` ), Habilidades ( `Skills` ) y Function Calling con Gemini

---

## 1. Objetivos de Aprendizaje

- **Evolución del Desarrollo Asistido por IA:** Comprender la transición desde la asistencia pasiva (*Ghost Text* `Tab` ) y la refactorización ( `Cmd+K` ) hacia la **Programación Basada en Agentes Autónomos** (*Agentic Coding*).
- **El Bucle Percepción - Acción - Verificación (PAV):** Analizar la arquitectura formal de agentes de código bajo el patrón ReAct (*Reasoning & Acting*).
- **Construcción Paso a Paso de un Agente con Gemini LLM:**
  - **Paso 1:** Definición de Reglas y Directivas de Comportamiento en la **raíz del proyecto** ( `.agents/rules/` ).
  - **Paso 2:** Creación de Habilidades Contextuales (*Skills* en `.agents/skills/` ).
  - **Paso 3:** Definición de Herramientas ejecutables (*Tools*) conectadas con el **servidor OAI-PMH REAL de Digital.CSIC** (visto en el Módulo 10).
  - **Paso 4:** Implementación de la clase `OpenScienceAgent` conectada con el **SDK oficial de Gemini** ( `google.generativeai` ) para ejecutar **Function Calling (Tool Calling)** en tiempo real.
  - **Paso 5:** Ejecución interactiva en vivo con publicaciones reales del CSIC y auditoría de trazabilidad.

## 2. De la Asistencia Pasiva a la Programación Basada en Agentes

A lo largo del curso hemos utilizado el autocompletado inteligente (*Ghost Text*) y la refactorización contextual ( `Cmd+K` ). Sin embargo, el desarrollo asistido por **Agentes Autónomos en Antigravity-IDE** permite delegar tareas complejas y multi-paso al modelo de lenguaje Gemini.

## Comparativa de Paradigmas en Antigravity-IDE

| Característica         | Autocompletado<br>Inline ( Tab ) | Refactorización<br>( Cmd+K )              | Agente Autónomo<br>( Antigravity Agent )                     |
|------------------------|----------------------------------|-------------------------------------------|--------------------------------------------------------------|
| Nivel de Autonomía     | Pasivo (Reactivo a la escritura) | Semi-autónomo (A nivel de bloque/archivo) | Alto (Autónomo en todo el proyecto)                          |
| Contexto de Análisis   | Línea actual / Archivo abierto   | Archivo actual / Selección                | Repositorio completo, logs, entorno virtual y herramientas   |
| Capacidad de Ejecución | Ninguna                          | Ninguna                                   | Ejecuta comandos shell, tests, herramientas y lee tracebacks |
| Flujo de Trabajo       | Sugerencia de código             | Edición directa de código                 | Bucle iterativo Percepción-Acción-Verificación (PAV)         |

### El Bucle Formal Percepción - Acción - Verificación (PAV)

Un agente inteligente no adivina respuestas; opera iterativamente sobre su entorno (el proyecto Python y las APIs de investigación) siguiendo un bucle de 3 etapas:

1. **Percepción ( $\mathcal{P}$ )**: El agente analiza la petición del usuario y examina el estado del entorno (archivos, respuestas de red, errores).
2. **Acción ( $\mathcal{A}$ )**: El agente selecciona y ejecuta una **Herramienta** (*Tool Call*) específica o edita el código.
3. **Verificación ( $\mathcal{V}$ )**: El agente comprueba los resultados mediante aserciones ( `assert` ), validez de formatos y control defensivo de errores, corrigiendo su curso si falla algo.

Entorno  $\mathcal{E}_t$   $\xrightarrow{\text{Observar}}$  Percepción  $\mathcal{P}_t$   $\xrightarrow{\text{Razonamiento Gemini}}$  Acción  $\mathcal{A}_t$   $\xrightarrow{\text{Ejecución Herramienta}}$   $\mathcal{V}_t$

## 3. Construcción del Agente Paso a Paso

### PASO 1: Configurar las Directivas y Reglas del Agente en la Raíz del Proyecto ( `.agents/` )

Los agentes leen sus reglas de comportamiento desde el directorio `.agents/` ubicado en la raíz del repositorio.

Podríamos crear el archivo manualmente o simplemente ejecutar la celda de abajo que escribe el archivo de reglas de comportamiento que va a tener el agente.

Determinaremos la raíz del espacio de trabajo y crearemos el archivo

`.agents/rules/curacion_abierta.md`:

```
In [1]: import os

# Determinar dinámicamente la raíz del espacio de trabajo
current_dir = os.getcwd()
project_root = os.path.abspath(os.path.join(current_dir, "..")) if os.path.t

# Definir la ruta exacta en la raíz del repositorio: .agents/rules/
rules_dir = os.path.join(project_root, ".agents", "rules")
os.makedirs(rules_dir, exist_ok=True)

#PRUEBA A PONERLE ALGUNA REGLA TUYA!!!

rule_content = """# Regla de Dominio: Curación de Publicaciones de Ciencia A

## Directivas Principales:
1. **Honestidad en Metadatos:** NUNCA inventar títulos, autores ni DOIs. Toc
2. **Verificación de Identificadores:** Validar siempre el formato de DOIs (
3. **Licencias Abiertas:** Promover el uso de licencias Creative Commons (CC
"""

rule_path = os.path.join(rules_dir, "curacion_abierta.md")
with open(rule_path, "w", encoding="utf-8") as f:
    f.write(rule_content)

print(f"✅ PASO 1 Completado: Regla creada en la RAÍZ del proyecto: '{rule_p
```

✅ PASO 1 Completado: Regla creada en la RAÍZ del proyecto: '/Users/linan/Desktop/RESEARCH/FORMACION/PYTHON\_PARA\_CIENCIA\_ABIERTA\_2/.agents/rules/curacion\_abierta.md'

## PASO 2: Definir una Habilidad Contextual en la Raíz (`.agents/skills/`)

Las **Habilidades (Skills)** son paquetes reutilizables situados en

`.agents/skills/<nombre_skill>/SKILL.md`.

Contienen la cabecera **YAML** (`name`, `description`) y el protocolo de actuación:

```
In [2]: # Definir la ruta en la raíz del proyecto: .agents/skills/open-science-curat
skill_dir = os.path.join(project_root, ".agents", "skills", "open-science-cu
os.makedirs(skill_dir, exist_ok=True)

skill_content = """___
name: open-science-curator
description: Instrucciones especializadas para consultar el servidor OAI-PMH
___

# Skill: Curador de Ciencia Abierta (CSIC)
```

```

## Protocolo de Actuación:
1. Recibir identificador persistente (DOI o Handle de Digital.CSIC).
2. Validar sintaxis con expresiones regulares (`re`).
3. Realizar petición HTTP al servidor OAI-PMH de Digital.CSIC (`https://digi
4. Extraer elementos Dublin Core (`dc:title`, `dc:creator`, `dc:date`, `dc:r
5. Auditar concordancia con la política de Acceso Abierto del CSIC.
6. Formatear la cita resultante en BibTeX para manuscritos en LaTeX.
"""

skill_path = os.path.join(skill_dir, "SKILL.md")
with open(skill_path, "w", encoding="utf-8") as f:
    f.write(skill_content)

print(f"✅ PASO 2 Completado: Habilidad de curación creada en la RAÍZ del pr

```

✅ PASO 2 Completado: Habilidad de curación creada en la RAÍZ del proyecto:  
 '/Users/linan/Desktop/RESEARCH/FORMACION/PYTHON\_PARA\_CIENCIA\_ABIERTA\_2/.agen  
 ts/skills/open-science-curator/SKILL.md'

## 📌 PASO 3: Definir las Herramientas Ejecutables ( Tools ) con Consulta REAL a Digital.CSIC

Las **Herramientas (Tools)** son las funciones Python que el modelo **Gemini** invocará dinámicamente mediante **Function Calling**.

Utilizaremos `requests` y `BeautifulSoup` ( xml parser) para realizar una **CONSULTA REAL** al servidor OAI-PMH de Digital.CSIC (`https://digital.csic.es/dspace-oai/request`), tal como se aprendió en el **Módulo 10 ( 10\_USANDO\_API.ipynb )**:

```

In [3]: import re
import requests
from bs4 import BeautifulSoup
from typing import Dict, Any, List, Optional

# --- HERRAMIENTA 1: Validación Sintáctica de Identificadores (DOI / Handle)
def tool_validate_identifier(identifier: str) -> Dict[str, Any]:
    """
    Valida si la cadena ingresada es un DOI estándar o un Handle de Digital.
    """
    clean_id = identifier.strip()

    # Patrón DOI estándar (ej. 10.1038/s41586-024-00001-x)
    # Expresión regular que valida la norma ISO 26324 para DOIs:
    # ^10.      -> Empieza obligatoriamente por el prefijo internacional '1
    # \d{4,9}/  -> Entre 4 y 9 dígitos del emisor (ej. 1038=Nature, 1016=El
    # [-._;()/:A-Za-z0-9]+$ -> Sufijo del artículo con caracteres alfanun
    doi_pattern = r"^10\.\d{4,9}/[-._;()/:A-Za-z0-9]+$"
    # Patrón Handle de Digital.CSIC (ej. 10261/280145 o 10261/2008)
    # Empieza por 10261, le sigue la /, y luego un numero.
    handle_pattern = r"^10261/\d+$"

    if re.match(handle_pattern, clean_id):

```

```

        return {
            "valid": True,
            "type": "Handle (Digital.CSIC)",
            "handle": clean_id,
            "oai_identifier": f"oai:digital.csic.es:{clean_id}",
            "url": f"https://hdl.handle.net/{clean_id}"
        }
    elif re.match(doi_pattern, clean_id):
        return {
            "valid": True,
            "type": "DOI",
            "doi": clean_id,
            "url": f"https://doi.org/{clean_id}"
        }
    else:
        return {
            "valid": False,
            "type": "Desconocido",
            "error": f"El identificador '{clean_id}' no cumple la sintaxis c
        }

# --- HERRAMIENTA 2: Consulta REAL al Servidor OAI-PMH de Digital.CSIC ---
def tool_fetch_digital_csic_real(handle_id: str, timeout: int = 15) -> Dict[
    """
    Realiza una consulta HTTP REAL al protocolo OAI-PMH de Digital.CSIC para
    Utiliza BeautifulSoup para parsear el XML Dublin Core (oai_dc).
    """
    clean_handle = handle_id.replace("https://hdl.handle.net/", "").replace(
    if not clean_handle.startswith("10261/"):
        clean_handle = f"10261/{clean_handle}"

    oai_url = "https://digital.csic.es/dspace-oai/request"
    oai_identifier = f"oai:digital.csic.es:{clean_handle}"
    params = {
        "verb": "GetRecord",
        "metadataPrefix": "oai_dc",
        "identifier": oai_identifier
    }

    try:
        response = requests.get(oai_url, params=params, timeout=timeout)
        if response.status_code != 200:
            return {"found": False, "error": f"Error HTTP {response.status_c

        soup = BeautifulSoup(response.text, "xml")

        error_tag = soup.find("error")
        if error_tag:
            return {"found": False, "error": f"Digital.CSIC OAI-PMH Error: {

        title_tag = soup.find("dc:title")
        title = title_tag.text.strip() if title_tag else "Sin título declara

        creators = [c.text.strip() for c in soup.find_all("dc:creator")]
        dates = [d.text.strip() for d in soup.find_all("dc:date")]

```

```

rights = [r.text.strip() for r in soup.find_all("dc:rights")]
subjects = [s.text.strip() for s in soup.find_all("dc:subject")]
publishers = [p.text.strip() for p in soup.find_all("dc:publisher")]

year = dates[0][:4] if dates else "2026"

return {
    "found": True,
    "data": {
        "handle": clean_handle,
        "title": title,
        "authors": creators if creators else ["Autor Institucional (C)"],
        "year": year,
        "publisher": publishers[0] if publishers else "Digital.CSIC",
        "rights": rights,
        "subjects": subjects[:5],
        "url": f"https://hdl.handle.net/{clean_handle}"
    }
}
except Exception as e:
    return {"found": False, "error": f"Excepción de conexión HTTP: {str(e)}"}

# --- HERRAMIENTA 3: Auditoría de Licencia de Acceso Abierto ---
def tool_audit_open_license(rights_list: List[str]) -> Dict[str, Any]:
    """
    Audita la lista de elementos dc:rights declarados en el registro OAI-PMH
    """
    rights_str = " ".join(rights_list).lower()

    is_open = False
    status_msg = "Licencia no identificada"

    if "openaccess" in rights_str or "open access" in rights_str or "info:eu
        is_open = True
        status_msg = "Acceso Abierto Confirmado (EU Semantics / Open Access)"
    elif "creative commons" in rights_str or "by" in rights_str:
        is_open = True
        status_msg = "Acceso Abierto (Licencia Creative Commons)"
    elif "restricted" in rights_str or "closed" in rights_str:
        is_open = False
        status_msg = "Acceso Restringido o Embargado"
    else:
        is_open = True
        status_msg = "Depositado en Digital.CSIC (Consulta Institucional)"

    return {
        "is_open_access": is_open,
        "rights_declared": rights_list,
        "status_summary": status_msg
    }

# --- HERRAMIENTA 4: Generador de Citas BibTeX para LaTeX ---
def tool_generate_bibtex(metadata: Dict[str, Any]) -> str:
    """

```

```

Genera una cita estructurada BibTeX a partir de metadatos reales de Digi
"""
data = metadata.get("data", metadata)
authors = data.get("authors", ["CSIC"])
authors_formatted = " and ".join(authors)

first_author_clean = authors[0].split(",")[0].lower().replace(" ", "")
year = data.get("year", "2026")
cite_key = f"{first_author_clean}{year}csic"

bibtex = f"""@article{{{cite_key}},
author      = {{{authors_formatted}}},
title       = {{{data.get('title', 'Sin título')}}},
journal      = {{{data.get('publisher', 'Digital.CSIC Repository')}}},
year         = {{{year}}},
url          = {{{data.get('url', '')}}}
}}}"""

return bibtex

print("✅ PASO 3 Completado: 4 Herramientas ejecutable registradas para Func

```

✅ PASO 3 Completado: 4 Herramientas ejecutable registradas para Function Calling con Gemini.

## 📌 PASO 4: Construir la Clase `OpenScienceAgent` con el SDK de Gemini ( `google-genai` )

Ahora encapsularemos la lógica en la clase `OpenScienceAgent` , utilizando el SDK oficial `google-genai` :

1. **Lectura de `.agents/`** : El agente lee las directivas de `.agents/rules/curacion_abierta.md` y `.agents/skills/open-science-curator/SKILL.md` , configurándolas como `system_instruction` del modelo.
2. **Registro de Herramientas:** Pasa las funciones Python en la lista `tools=[...]` a `GenerateContentConfig` .
3. **Llamadas a Gemini ( `gemini-2.5-flash` ):** Gemini analiza el prompt del usuario, decide autónomamente qué herramientas invocar (*Function Calling*), recibe los resultados y compila el informe final.

```

In [4]: import json
from dotenv import load_dotenv
from google import genai
from google.genai import types

# 1. Cargar el archivo .env
env_loaded = load_dotenv()
# 2. Verificar que clave API específica se ha cargado
key_name = "GEMINI_API_KEY"
api_key = os.environ.get(key_name)
if api_key:
    # Enmascarar la clave para no exponerla completa en la pantalla (ej: AIz
    masked_key = f"{api_key[:2]}...{api_key[-3:]}"

```

```

    print(f"✅ Variable de entorno '{key_name}' cargada con éxito desde .env")
else:
    print(f"⚠️ No se encontró la variable '{key_name}' en el archivo .env")

```

✅ Variable de entorno 'GEMINI\_API\_KEY' cargada con éxito desde .env (AQ...n wA)

load\_dotenv() es una función de la librería python-dotenv (incluida en la línea 32 de nuestro requirements.txt).

Su objetivo fundamental es cargar de forma segura las variables de entorno confidenciales (como claves de API, contraseñas o tokens de acceso a bases de datos) desde un archivo local oculto llamado .env directamente hacia la memoria del sistema en Python (os.environ).

In [5]: *#El agente, creado como una clase que usa el SDK oficial de google-genai*

```

class OpenScienceAgent:
    """
    Agente Autónomo de Curación para Ciencia Abierta en el CSIC.
    Realiza llamadas REALES al modelo Gemini utilizando el SDK oficial `google-genai`
    y resolviendo Herramientas mediante Function Calling.
    """
    def __init__(self, agent_name: str = "CSIC-Gemini-Agent", model_name: str = "gemini-1.5-pro"):
        self.agent_name = agent_name
        self.model_name = model_name
        self.history = []

        # 1. Cargar explícitamente las Reglas desde .agents/rules/
        self.rules_path = os.path.join(project_root, ".agents", "rules", "rules.txt")
        self.rules_text = self._load_file_content(self.rules_path, fallback="")

        # 2. Cargar explícitamente la Habilidad desde .agents/skills/
        self.skill_path = os.path.join(project_root, ".agents", "skills", "skills.txt")
        self.skill_text = self._load_file_content(self.skill_path, fallback="")

        # Combinar Directivas y Habilidad como System Instruction para Gemini
        self.system_instruction = f"{self.rules_text}\n\n{self.skill_text}"

        # 3. Registrar Herramientas ejecutable para el LLM
        self.tools = [
            tool_validate_identifier,
            tool_fetch_digital_csic_real,
            tool_audit_open_license,
            tool_generate_bibtex
        ]

        # 4. Inicializar cliente Gemini si existe la API Key
        self.api_key = os.environ.get("GEMINI_API_KEY")
        if self.api_key:
            self.client = genai.Client(api_key=self.api_key)
            self._log(f"Agente '{self.agent_name}' listo con llamadas REALES")
        else:
            self.client = None

```

```

        self._log(f"⚠️ GEMINI_API_KEY no configurada. Habilitado modo re

self._log(f"📄 Reglas configuradas desde: '{self.rules_path}' ({len(
self._log(f"💡 Habilidad configurada desde: '{self.skill_path}' ({le

def _load_file_content(self, filepath: str, fallback: str) -> str:
    """Lee el contenido de un archivo en el espacio de trabajo."""
    if os.path.exists(filepath):
        with open(filepath, "r", encoding="utf-8") as f:
            return f.read()
    return fallback

def _log(self, message: str) -> None:
    """Mantiene la trazabilidad e historial del agente."""
    self.history.append(message)
    print(f"🤖 [{self.agent_name}] {message}")

def process_publication_handle(self, handle_input: str) -> Dict[str, Any]
    """
    Ejecuta una llamada al LLM Gemini pasando las instrucciones del sist
    """
    self._log(f"=== INICIANDO TAREA: Curación Abierta para Handle '{handle_input}'")

    prompt = f"""Por favor, cura la publicación de Digital.CSIC con Handle '{handle_input}'.
    Sigue las reglas del proyecto y el protocolo de la H
    1. Valida el formato del identificador '{handle_input}'.
    2. Obtén los metadatos reales del servidor OAI-PMH c
    3. Audita la licencia de acceso abierto.
    4. Genera la cita formateada en BibTeX para LaTeX."""

    if self.client:
        self._log(f"🧠 Enviando Prompt + System Instructions + 4 Tools a Gemini")
        config = types.GenerateContentConfig(
            system_instruction=self.system_instruction,
            tools=self.tools,
            temperature=0.1
        )
        try:
            response = self.client.models.generate_content(
                model=self.model_name,
                contents=prompt,
                config=config
            )
            self._log(f"✅ Respuesta recibida de Gemini con resolución automática")
            return {
                "status": "SUCCESS",
                "mode": "REAL_GEMINI_LLM",
                "model": self.model_name,
                "handle": handle_input,
                "llm_response": response.text,
                "applied_rule": "curacion_abierta.md",
                "applied_skill": "open-science-curator"
            }
        except Exception as e:
            self._log(f"⚠️ Error en la llamada a la API de Gemini: {e}.
```

```

# Modo respaldo si no hay API Key o falla la llamada HTTP a Gemini
val_res = tool_validate_identifier(handle_input)
if not val_res.get("valid"):
    return {"status": "ERROR", "reason": val_res.get("error")}
fetch_res = tool_fetch_digital_csic_real(val_res.get("handle", handle_input))
if not fetch_res.get("found"):
    return {"status": "ERROR", "reason": fetch_res.get("error")}
lic_res = tool_audit_open_license(fetch_res["data"]["rights"])
bib_code = tool_generate_bibtex(fetch_res["data"])

return {
    "status": "SUCCESS",
    "mode": "DIRECT_TOOL_ORCHESTRATION",
    "applied_rule": "curacion_abierta.md",
    "applied_skill": "open-science-curator",
    "handle_info": val_res,
    "metadata_real": fetch_res["data"],
    "license_audit": lic_res,
    "bibtex": bib_code
}

def generate_summary_report(self, result: Dict[str, Any]) -> None:
    """
    Imprime el informe final de la tarea de curación.
    """
    if result.get("status") != "SUCCESS":
        print(f"\n❌ ERROR EN LA TAREA DEL AGENTE: {result.get('reason')}")
        return

    print("\n--- INFORME FINAL GENERADO POR EL AGENTE ---")
    print(f"📄 Regla Aplicada: {result.get('applied_rule')}")
    print(f"💡 Habilidad Aplicada: {result.get('applied_skill')}")
    print(f"⚙️ Modo de Ejecución: {result.get('mode')}")

    if result.get("mode") == "REAL_GEMINI_LLM":
        print(f"🤖 Modelo Gemini Utilizado: {result.get('model')}")
        print("\n--- RESPUESTA AUTÓNOMA DE GEMINI (TOOL CALLING) ---")
        print(result.get("llm_response"))
    else:
        meta = result["metadata_real"]
        lic = result["license_audit"]
        print(f"📌 Handle Institucional: {meta.get('handle')}")
        print(f"🔗 Enlace Oficial: {meta.get('url')}")
        print(f"📖 Título Real: {meta.get('title')}")
        print(f"👤 Autores CSIC: {' '.join(meta.get('authors', []))}")
        print(f"📅 Año: {meta.get('year')}")
        print(f"📄 Licencia: {lic.get('status_summary')}")
        print("\n--- CITA BIBTEX COMPILADA ---")
        print(result.get('bibtex'))

```

In [6]: `agent = OpenScienceAgent(agent_name="CSIC-Gemini-Agent")`

🤖 [CSIC-Gemini-Agent] Agente 'CSIC-Gemini-Agent' listo con llamadas REALES a Gemini (gemini-2.5-flash).

🤖 [CSIC-Gemini-Agent] 📄 Reglas configuradas desde: '/Users/linan/Desktop/RESEARCH/FORMACION/PYTHON\_PARA\_CIENCIA\_ABIERTA\_2/.agents/rules/curacion\_abierta.md' (538 bytes).

🤖 [CSIC-Gemini-Agent] 💡 Habilidad configurada desde: '/Users/linan/Desktop/RESEARCH/FORMACION/PYTHON\_PARA\_CIENCIA\_ABIERTA\_2/.agents/skills/open-science-curator/SKILL.md' (770 bytes).

## 📌 PASO 5: Pruebas de Ejecución en Vivo con Publicaciones Reales de Digital.CSIC

Ejecutaremos el agente con los Handles reales e imprimiremos el informe con

```
agent.generate_summary_report(result) :
```

```
In [10]: handles = [  
        "10261/163587",  
        "10261/187350",  
        "10261/401353"]
```

```
In [11]: import time  
        from lib.mylib import linea  
        for handle in handles:  
            time.sleep(1) #Le damos aire y no agobiamos al servidor ;)  
            # Petición REAL a Digital.CSIC  
            linea("*",40)  
            result = agent.process_publication_handle(handle)  
            agent.generate_summary_report(result)  
            linea("-",40)
```

\*\*\*\*\*

🤖 [CSIC-Gemini-Agent] === INICIANDO TAREA: Curación Abierta para Handle '10261/163587' ===

🤖 [CSIC-Gemini-Agent] 🧠 Enviando Prompt + System Instructions + 4 Tools a Gemini (gemini-2.5-flash)...

🤖 [CSIC-Gemini-Agent] ✅ Respuesta recibida de Gemini con resolución autónoma de Herramientas.

--- INFORME FINAL GENERADO POR EL AGENTE ---

📄 Regla Aplicada: curacion\_abierta.md

💡 Habilidad Aplicada: open-science-curator

⚙️ Modo de Ejecución: REAL\_GEMINI\_LLM

🤖 Modelo Gemini Utilizado: gemini-2.5-flash

--- RESPUESTA AUTÓNOMA DE GEMINI (TOOL CALLING) ---

La publicación de Digital.CSIC con Handle '10261/163587' ha sido curada exitosamente:

1. **\*\*Validación del Identificador:\*\*** El Handle '10261/163587' es un identificador válido de Digital.CSIC.

2. **\*\*Metadatos Obtenidos:\*\***

\* **\*\*Título:\*\*** El axioma del bien común como fundamento de los derechos naturales del hombre

\* **\*\*Autor(es):\*\*** Peña, Lorenzo

\* **\*\*Año:\*\*** 2018

\* **\*\*Editor:\*\*** Tirant lo Blanch

\* **\*\*Derechos:\*\*** info:eu-repo/semantics/openAccess

\* **\*\*URL:\*\*** https://hdl.handle.net/10261/163587

3. **\*\*Auditoría de Licencia:\*\*** La publicación declara "info:eu-repo/semantics/openAccess", confirmando su estado de Acceso Abierto.

4. **\*\*Cita BibTeX:\*\***

```bibtex

@misc{Pena2018Axioma,

author = {Peña, Lorenzo},

title = {El axioma del bien común como fundamento de los derechos naturales del hombre},

year = {2018},

publisher = {Tirant lo Blanch},

howpublished = {\url{https://hdl.handle.net/10261/163587}},

note = {Digital.CSIC}

}

```

\*\*\*\*\*

🤖 [CSIC-Gemini-Agent] === INICIANDO TAREA: Curación Abierta para Handle '10261/187350' ===

🤖 [CSIC-Gemini-Agent] 🧠 Enviando Prompt + System Instructions + 4 Tools a Gemini (gemini-2.5-flash)...

🤖 [CSIC-Gemini-Agent] ⚠️ Error en la llamada a la API de Gemini: 429 RESOURCE\_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n\* Quota exceeded for metric: generativelanguage.googleapis.com/generate\_content\_free\_tier\_requests, limit: 5, model: gemini-2.5-flash\nPlease retry in 30.341906294

```
s.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerMinutePerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-2.5-flash'}, 'quotaValue': '5'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '30s'}]}. Pasando a respaldo directo.
```

--- INFORME FINAL GENERADO POR EL AGENTE ---

 Regla Aplicada: curacion\_abierta.md  
 Habilidad Aplicada: open-science-curator  
 Modo de Ejecución: DIRECT\_TOOL\_ORCHESTRATION  
 Handle Institucional: 10261/187350  
 Enlace Oficial: <https://hdl.handle.net/10261/187350>  
 Título Real: Benefits and costs of renewable electricity in Europe  
 Autores CSIC: Ortega Izquierdo, Margarita, Río González, Pablo del  
 Año: 2019  
 Licencia: Acceso Restringido o Embargado

--- CITA BIBTEX COMPILADA ---

```
@article{ortegaizquierdo2019csic,  
  author    = {Ortega Izquierdo, Margarita and Río González, Pablo del},  
  title     = {Benefits and costs of renewable electricity in Europe},  
  journal   = {Elsevier},  
  year      = {2019},  
  url       = {https://hdl.handle.net/10261/187350}  
}
```

\*\*\*\*\*

 [CSIC-Gemini-Agent] === INICIANDO TAREA: Curación Abierta para Handle '10261/401353' ===

 [CSIC-Gemini-Agent]  Enviando Prompt + System Instructions + 4 Tools a Gemini (gemini-2.5-flash)...

 [CSIC-Gemini-Agent]  Error en la llamada a la API de Gemini: 429 RESOURCE\_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n\* Quota exceeded for metric: generativelanguage.googleapis.com/generate\_content\_free\_tier\_requests, limit: 5, model: gemini-2.5-flash\nPlease retry in 29.131502499 s.', 'status': 'RESOURCE\_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate\_content\_free\_tier\_requests', 'quotaId': 'GenerateRequestsPerMinutePerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-2.5-flash', 'location': 'global'}, 'quotaValue': '5'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '29s'}]}. Pasando a respaldo directo.

--- INFORME FINAL GENERADO POR EL AGENTE ---

 Regla Aplicada: curacion\_abierta.md  
 Habilidad Aplicada: open-science-curator  
 Modo de Ejecución: DIRECT\_TOOL\_ORCHESTRATION

 Handle Institucional: 10261/401353  
 Enlace Oficial: <https://hdl.handle.net/10261/401353>  
 Título Real: Shining disco: shedding light into the systematics of the family Discodorididae (Gastropoda: Nudibranchia) [Dataset]  
 Autores CSIC: Fernández-Vilert, Robert, Arnedo, Miquel A., Salvador, Xavier, Valdés Gallego, Ángel, Schrödl, Michael, Moles, Juan  
 Año: 2025  
 Licencia: Acceso Abierto Confirmado (EU Semantics / Open Access)

--- CITA BIBTEX COMPILADA ---

```
@article{fernández-vilert2025csic,  
  author    = {Fernández-Vilert, Robert and Arnedo, Miquel A. and Salvador,  
Xavier and Valdés Gallego, Ángel and Schrödl, Michael and Moles, Juan},  
  title     = {Shining disco: shedding light into the systematics of the fam  
ily Discodorididae (Gastropoda: Nudibranchia) [Dataset]},  
  journal   = {Linnean Society of London},  
  year      = {2025},  
  url       = {https://hdl.handle.net/10261/401353}  
}
```

## Verificación Defensiva ante Identificadores Inválidos

Probemos cómo actúa el agente ante un identificador erróneo o inexistente.

```
In [12]: # Prueba defensiva con un identificador erróneo  
print("=====")  
result_error = agent.process_publication_handle("HANDLE_INEXISTENTE_999")  
agent.generate_summary_report(result_error)
```

```
=====
```

 [CSIC-Gemini-Agent] === INICIANDO TAREA: Curación Abierta para Handle 'HANDLE\_INEXISTENTE\_999' ===

 [CSIC-Gemini-Agent]  Enviando Prompt + System Instructions + 4 Tools a Gemini (gemini-2.5-flash)...

 [CSIC-Gemini-Agent]  Respuesta recibida de Gemini con resolución autónoma de Herramientas.

--- INFORME FINAL GENERADO POR EL AGENTE ---

 Regla Aplicada: curacion\_abierta.md  
 Habilidad Aplicada: open-science-curator  
 Modo de Ejecución: REAL\_GEMINI\_LLM  
 Modelo Gemini Utilizado: gemini-2.5-flash

--- RESPUESTA AUTÓNOMA DE GEMINI (TOOL CALLING) ---

El identificador 'HANDLE\_INEXISTENTE\_999' no es un Handle de Digital.CSIC válido ni un DOI. Por favor, proporciona un identificador con el formato correcto (por ejemplo, '10261/XXXXX' para Handles o '10.XXXX/XXXXX' para DOIs) para que pueda curar la publicación.

## 4. Resumen del Módulo y Sigüientes Pasos

En este módulo hemos abordado de forma práctica y estructurada el **desarrollo asistido por agentes con Gemini LLM en Antigravity-IDE**:

1. **Del Autocompletado Pasivo a los Agentes:** Comprendimos las diferencias entre *Ghost Text Tab* , refactorización `Cmd+K` y orquestación autónoma mediante el ciclo **Percepción - Acción - Verificación (PAV)**.
  2. **Paso 1 (Reglas en `.agents/rules/` ):** Definimos el marco ético y técnico del agente en la raíz del proyecto ( `.agents/rules/curacion_abierta.md` ).
  3. **Paso 2 (Habilidades en `.agents/skills/` ):** Empaquetamos conocimiento experto en `.agents/skills/open-science-curator/SKILL.md` .
  4. **Paso 3 (Herramientas `Tools` Reales):** Implementamos funciones Python que realizan **consultas HTTP REALES al servidor OAI-PMH de Digital.CSIC** (reutilizando los conocimientos del Módulo 10).
  5. **Paso 4 & 5 (Agente Gemini con `google.genai` ):** La clase `OpenScienceAgent` configura Gemini enviando el System Instruction y la lista de Tools para ejecutar **Function Calling / Tool Calling** en tiempo real.
- 

## **Siguiente Módulo: Proyecto Final de GitHub**

En el próximo y último módulo del curso (**Módulo 14: Proyecto Final de GitHub**), integraremos todo el conocimiento del curso: estructuras de datos, modularización, manipulación de metadatos con Pandas, gráficos en Matplotlib, empaquetado de librerías, integración continua (CI/CD) y desarrollo guiado por agentes en GitHub.